

Lab 2 - EECS 211

Winter 2026

1 Overview

In this lab, you will learn how to design and implement a software interrupt for the QEMU processor. A software interrupt is a way for a program to request the operating system or supervisor program to perform a specific service on its behalf. In this lab, we will create a custom software interrupt and demonstrate how it can be used to perform a simple task.

Lab environment: All tasks in this lab are performed on EECS-provided Linux servers.

Readings and references: Students are encouraged to consult the following resources for additional background:

- Chapter 35 of RISC-V Instruction Set Manual - This chapter contains a summary of the different RISC-V assembly language instructions.
- RISC-V Assembly Language Reference Manual – A PDF document that explains RISC-V assembly language instructions, registers, and usage patterns in detail.

Rubrics (100 points):

- **PDF report:** This includes a screenshot of the messages (and the PC addresses) displayed in the terminal and a brief explanation of your implementation choices.
- **trap.s:** All the TODOs in the template should be completed.

You need to display the following three things. Note that displaying the PC address is optional and for extra credit since the conversion to the ASCII character is a bit complicated and not the main goal of this lab.

- In TODO_2: "Before trap handler" message.
- In TODO_5: "Inside trap handler" message.
- In TODO_4: "After trap handler" message.

Note: TODOs 1 and 6 are weighted at 20 points each and all other TODOs are worth 10 points each. Displaying the PC address is optional and for extra credit. Points will be deducted if no screenshots or sufficient explanations are given. The students should also **fill the honesty statement and upload it to canvas**, otherwise 30 points will be deducted.

Due date: 01/28/2026, 23:59:59 PST

2 Implementation Steps

Implementing a software interrupt on a QEMU processor using the RISC-V RV64 architecture involves several key steps:

1. **Define a Machine Trap Vector:** Define a custom machine trap vector (`mtvec`) in your assembly code. This vector contains the address of the interrupt handler routine that will be executed when the software interrupt is triggered. We will use the `trap_entry` label for this.

2. **Invoke the Software Interrupt:** Use the `ecall` instruction that QEMU recognizes to trigger the software interrupt.
3. **Implement the Interrupt Handler Routine:** Define the interrupt handler routine at the address specified in the custom trap vector. Inside the routine, perform the desired operations, computations, or actions (e.g., displaying messages).
4. **Store the Result:** After performing operations in the interrupt handler routine, store the result in a designated memory location or register and return to the original execution point. We will use the `mret` instruction for this.
5. **Halt the Program:** After completing all operations, use a loop to halt the program like we did in Lab 1.

3 Example of RISC-V Assembly Code

The provided RISC-V assembly code listens for keystrokes, modifies the data (shifts the character), and prints the result.

```

1 .global _start
2
3 _start:
4     li s0, 0x10000000      # UART base address
5
6 listen_loop:
7     # Check if a key was pressed
8     lb t0, 5(s0)           # load line status register (offset 5)
9     andi t0, t0, 1         # check bit 0 (data ready)
10    beqz t0, listen_loop    # if 0, jump back and check again
11
12    # Read the character
13    lb a0, 0(s0)           # load the character from the data register
14
15    # Encrypt the character
16    addi a0, a0, 1         # add 1 to the ASCII value
17
18    # Print the new character
19    sb a0, 0(s0)           # store the modified character back to UART
20
21    # Repeat
22    j listen_loop          # jump back to listen for the next key

```

4 Assignment: Implement the Custom Interrupt Handler

4.1 Objective:

Write a RISC-V assembly program that demonstrates the use of a software trap to handle an interrupt. Your program should gracefully handle the exception and print appropriate messages and program counter (PC) addresses depending on where it is executing in your code. The provided template consists of two main sections: the main code and the interrupt code.

4.1.1 Main Code

The main code is the central part of the program that performs various operations before triggering a custom software interrupt.

- Load the address of the machine trap vector into the `mtvec` register.
- Invoke a custom software interrupt by triggering the trap handler using the `ecall` instruction.
- Resume execution after returning from the interrupt handler.

4.1.2 Interrupt Handler

The interrupt code is executed when the custom trap handler is triggered.

- Save the program state by pushing all the registers to the stack.
- Call the interrupt handler corresponding to the type of interrupt.
- Restore the program state by popping all registers from the stack.
- Return to the address of the main code by calling `mret`.

4.2 Code Template (trap.s)

You will need to implement the TODOs in the template code to make it work.

```
1 .global _start
2
3 _start:
4     # TODO 1:
5     # 1. Load the trap_entry address into a register
6     # 2. Write it into the control status register (CSR), mtvec using the csrw
       instruction
7
8     # init stack pointer
9     la sp, stack_start # points to the start of the stack, see linker script.
10
11     # TODO_2:
12     # 1. Display a message in the terminal before triggering the trap handler
13     # 2. (Optional) Display the current program counter value
14
15     # TODO_3:
16     # 1. Trigger a software interrupt with the ecall instruction
17
18     # TODO_4:
19     # 1. Display a message in the terminal after coming back from the trap
       handler
20     # 2. (Optional) Display the current program counter value
21     # 3. Halt the program using a loop
22
23 handle_trap:
24     # TODO_5:
25     # 1. Display a message in the terminal that tells the program is inside the
       trap handler
26     # 2. (Optional) Display the current program counter value
27
28     # TODO_6:
29     # 1. Load the return address from the mepc reg
30     # 2. Increment the address by one instruction
31     # 3. Save the updated address in the mepc reg
32     # 4. Jump back to the trap_entry
33
34 .align 2
35 trap_entry:
36     # store state before handling the trap
37     sd ra, 0*8(sp)
38     sd a0, 1*8(sp)
39     sd a1, 2*8(sp)
40     sd a2, 3*8(sp)
41     sd a3, 4*8(sp)
42     sd a4, 5*8(sp)
43     sd a5, 6*8(sp)
44     sd a6, 7*8(sp)
45     sd a7, 8*8(sp)
46     sd t0, 9*8(sp)
```

```

47     sd t1, 10*8(sp)
48     sd t2, 11*8(sp)
49     sd t3, 12*8(sp)
50     sd t4, 13*8(sp)
51     sd t5, 14*8(sp)
52     sd t6, 15*8(sp)
53     addi sp, sp, 17*8
54
55     # TODO_7:
56     # 1. Jump to the handle_trap
57
58     # load state after handling the trap
59     addi sp, sp, -17*8
60     ld ra, 0*8(sp)
61     ld a0, 1*8(sp)
62     ld a1, 2*8(sp)
63     ld a2, 3*8(sp)
64     ld a3, 4*8(sp)
65     ld a4, 5*8(sp)
66     ld a5, 6*8(sp)
67     ld a6, 7*8(sp)
68     ld a7, 8*8(sp)
69     ld t0, 9*8(sp)
70     ld t1, 10*8(sp)
71     ld t2, 11*8(sp)
72     ld t3, 12*8(sp)
73     ld t4, 13*8(sp)
74     ld t5, 14*8(sp)
75     ld t6, 15*8(sp)
76
77     # TODO_8:
78     # 1. Return to the main code using the mret instruction
79
80 .section .data
81
82 before:
83     .string "Before trap handler\n"
84
85 inside:
86     .string "Inside trap hander\n"
87
88 after:
89     .string "After trap handler\n"

```